Reversed: webhook-per-call, replaced by event sourcing

380 webhook events in 90 minutes took down the practice's n8n box, a 1 GB VPS their IT had spun up for the integration. Replacing the bus with an append-only log took 240 lines and two and a half days.

28 MAY 2026 · 6 MIN READ

The first version of the voice agent fanned out one webhook per call event: call started, call ended, escalation, transcription chunk, intent confidence drop, every one a separate HTTP request to the practice's downstream system. On day 28 of pilot two, the 8 to 10 am peak collapsed it. Here is what the failure looked like and what ships now.

What shipped first

The original design was the obvious one. Every meaningful thing that happened during a call became a webhook POST to whatever the practice had pointed it at: their PMS integration server, an n8n bridge, sometimes just a Zapier catcher. Five event types per call on average, sometimes twelve when a long conversation produced a lot of intent drift. Each event carried its full payload so consumers would not have to call back for context.

Why this looked right at the time: the consumer was usually a script the practice's IT contact owned, and "you get a JSON POST when something happens" is the lowest-friction integration story you can ship. The first two pilots ran on it for six weeks without incident.

The signal that forced the reversal

Pilot two had a busy Tuesday. Sixty-something incoming calls between 8 and 10 am, which is normal for a six-doctor general practice the week after a bank holiday. Somewhere around 380 webhook events went into their n8n bridge in 90 minutes. n8n was on a 1 GB Hetzner cloud VPS the practice's IT had spun up for the integration. It crawled, then it stopped accepting connections.

The bus did exactly what it was built to do: retried each failed webhook with exponential backoff, kept the in-memory queue of pending events, and continued trying for 90 minutes. When n8n came back the queue flushed in thirty seconds and overran it a second time. The practice did not lose calls because the voice agent itself does not depend on the webhook bus; what they lost was the calendar sync, which is most of the point.

The post-mortem identified three things:

- The bus had no durable record on the sending side. If that process restarted, the queue was gone.
- Consumers were forced to be online and fast at exactly the moment they were most likely to be neither.
- Event ordering was best-effort, not guaranteed. A retry could land call_ended before escalation.

What runs now

Events go to a durable append-only log first, persisted to Postgres with a monotonic sequence number per call. Consumers are projections that read from that log at their own pace and remember the last sequence they processed. Nothing is "delivered" in the webhook sense; everything is "available to pull". When the practice's n8n is up, it pulls. When it is down, the log sits there. When it comes back, it resumes from its cursor.

The migration was less dramatic than it sounds because the log *is* the Postgres table that already existed for debug tooling. What got added was a cursor table per consumer and a small Edge Function that does the pull. Total code shipped: about 240 lines plus a 60-line migration. Consumer migration on the practice side was a 20-minute n8n change: "poll this endpoint" instead of "listen for POST".

The side benefit nobody asked for but everyone notices: full replay. New consumer joins, it replays from sequence 0, ends up with the same state the old consumers have. No "the events from last Tuesday never arrived" conversations. The audit trail for DSGVO purposes (which events happened, in what order, when was the practice told) is the log itself, no reconstruction needed.

What it cost

Two and a half engineering days. One day to write the log and the cursor table. Half a day for the pull endpoint. One day for the two practices on the bus at the time to migrate their n8n. The retry storm itself had cost a four-hour incident debrief with the practice and a slightly delayed Wednesday lunch, so the project paid back inside one bad morning.

Where this still does not apply

If your consumer count is one and the consumer is your own backend, the webhook-bus pattern is fine and event sourcing is overkill. The reversal paid off because there were two practices, two integration shapes, and a foreseeable third one. The trigger for the pattern is "the second consumer has different uptime characteristics than the first", not call volume in absolute terms.

The lesson

A webhook bus assumes the consumer's uptime is your problem. An append-only log assumes the consumer's uptime is the consumer's problem. The second assumption is the one a small studio can afford to hold. Build the bus that is forgiving of the practice's IT contact being on holiday, not the one that depends on them not being.

© 2026 Fabian Ilg. Licensed CC BY-NC-ND 4.0. Canonical copy: https://github.com/Fabi-SPL/essays